Práctica 7 de Técnicas Físicas III: Modelo de Ising Gauge

Julen Bermejo, Paula Otal, Ricardo Sáez

Primer cuatrimestre 2024

Índice

1. Introducción **3**

2. Programación de la red **3**

3. Test de verificación **5**

4. Algoritmo de metrópolis y termalización **8**

5. Comprobación de las leyes del área y el perímetro **13**

6. Conclusiones **17**

A. Tratamiento del error: **18**

B. Código Comprobación Ley del Área y del Perímetro **21**

C. Código de Python utilizado para el ajuste funcional: **22**

D. Datos exactos de las figuras: **24**

1. Introducción

El objetivo de esta práctica es comprobar las leyes del área y del perímetro para una red particular usando el modelo de Ising gauge. Para ello, se realizará una simulación de una red cúbica con las condiciones periódicas del modelo y se realizará una simulación de Monte Carlo para obtener el resultado.

El código se realizará en dos partes, diferenciadas por su lenguaje de programación. La primera será en C para la realización de la simulación y la obtención de los valores experimentales. La otra se esribirá en Python para realizar el análisis de datos y obtener las gráficas necesarias para las comprobaciones del modelo, junto con el cálculo de errores.

2. Programación de la red

El modelo de Ising gauge 3D consiste en una red tridimensional de puntos (partículas) equiespaciadas unidas por aristas según su espín. Su hamiltoniano (y por tanto su energía) viene dada por:

$$\mathcal{H} = -J \sum_{p} \sigma_p - H \sum_{\nu,\mu} \sigma_{\nu,\mu} \tag{1}$$

siendo σ el espín (que para cada arista solo puede tomar valores 1 o -1), p cada plaqueta de la red (que se define como el producto de espines de las aristas de la plaqueta), (ν, μ) cada arista de la red, H el campo magnético externo y J un valor constante. Por simplicidad, asumiremos que $J = 1$ y que no hay ningún campo magnético externo. Es decir, la energía se reduce a la suma del espín de cada plaqueta con signo cambiado.

Además, esta red es invariante gauge, es decir, bajo transformación gauge, no se debe ver afectado el valor de $\mathcal{H}$. Esto será importante para verficar que la red está bien definida.

Se procede a explicar el código de C para la simulación.

Se definen inicialmente las variables globales:

```
#define L 24

int red[L][L][L][3]
```

L aquí representa el lado de la red y la variable *red* define el espín de cada arista de la red. Normalmente, se mantendrá $L = 24$, pero para ciertas simulaciones de prueba del código se utilizará puntualmente $L = 10$. Para decidir cómo será su configuración inicial, se realiza una asignación aleatoria de espines mediante el subalgoritmo *inicializarRed*:

```
void inicializarRed() {
    for (int i = 0; i < L; i++) {
        for (int j = 0; j < L; j++) {
```

```
            for (int k = 0; k < L; k++) {
                for (int d = 0; d < 3; d++) {
                    red[i][j][k][d] = (frand() < 0.5) ? -1 : 1;
                }
            }
        }
    }
}
```

Este código asigna un valor aleatorio a cada componente de *red* entre 1 y -1 con la misma probabilidad. Notar que se llama a la función *frand* programada como:

```
double frand() {
    return (double)rand() / RAND_MAX;
}
```

que devuelve un número aleatorio entre 0 y 1 con una distribución de probabilidad uniforme. De manera auxiliar, se utilizará la función *inicializarRedMin* que devuelve la situación de mínima energía, es decir, con todos los espines iguales a 1:

```
void inicializarRedMin(){
for (int i = 0; i < L; i++) {
    for (int j = 0; j < L; j++) {
        for (int k = 0; k < L; k++) {
            for (int d = 0; d < 3; d++) {
                red[i][j][k][d] = 1;
                }
            }
        }
    }
}
```

Este código, sin embargo, se utilizará más con la idea de comparar valores y observar casos extremos en distintos apartados.

Se pasa al cálculo de la energía de la red. Como esta se puede calcular como la suma del espín de cada plaqueta, se creará una función auxiliar *energiaPlaqueta* que, dadas la coordenadas de un punto (x, y, z) y una dirección (d), devuelva la energía de la plaqueta, calculada como el producto de las cuatro aristas que la conforman:

```
int energiaPlaqueta(int x, int y, int z, int d) {
int suma = 0;
if (d == 0) { // x
    //printf("%d %d %d %d\n",red[x][y][z][2],red[x][y][z][0],
        red[(x+1)%L][y][z][2],red[x][y][(z+1)%L][0]);
    suma = red[x][y][z][2]*red[x][y][z][0]*
        red[(x+1)%L][y][z][2]*red[x][y][(z+1)%L][0];
}
else if (d == 1) { // y
```

```
        //printf("%d %d %d %d\n",red[x][y][z][0],red[(x+1)%L][y][z][1],
            red[x][y][z][1],red[x][(y+1)%L][z][0]);
        suma = red[x][y][z][0]*red[(x+1)%L][y][z][1]*
            red[x][y][z][1]*red[x][(y+1)%L][z][0];
    }
    else if (d == 2) { // z
        //printf("%d %d %d %d\n",red[x][y][z][2],red[x][y][z][1],
            red[x][(y+1)%L][z][2],red[x][y][(z+1)%L][1]);
        suma = red[x][y][z][2]*red[x][y][z][1]
            *red[x][(y+1)%L][z][2]*red[x][y][(z+1)%L][1];
    }
    return suma;
}
```

Las líneas comentadas sirven como comprobación del funcionamiento del código para poder calcular los valores a mano de estas energías y comprobar la utilidad de la función. Notar que el código define 3 casos distintos:

- $d = 0$: Se calcula la energía de la plaqueta en el plano XZ.
- $d = 1$: Se calcula la energía de la plaqueta en el plano XY.
- $d = 2$: Se calcula la energía de la plaqueta en el plano YZ.

Con esta función a mano, es sencillo realizar el cálculo de la energía de la red:

```
    int energiaRed(){
        int suma =0;
        for (int i = 0; i < L; i++) {
            for (int j = 0; j < L; j++) {
                for (int k = 0; k < L; k++) {
                    for (int d = 0; d < 3; d++) {
                        suma += energiaPlaqueta(i,j,k,d);
                        }
                    }
                }
            }
        return -suma;
    }
```

La función *energiaRed* simplemente recorre una red ya inicializada y suma las energías de todas las plaquetas en esta. Devuelve el negativo de la suma tal y como sugiere 1. Con esto, ya estaría programada la red, pero se realizarán unos tests para comprobar que la red cumple lo esperado.

3. Test de verificación

A medida que se avanza con la práctica, puede servir de ayuda realizar pruebas para comprobar que el código no falla. En esta sección se explicarán los test de verificación más significativos que se llevaron a cabo.

Ya programados los subalgoritmos *inicializarRed* y *energiaRed*, queremos comprobar si las energías asignadas son coherentes. Dado que *inicializarRed* utiliza un generador de números aleatorios con distribución uniforme, *frand*, se espera que, creando un gran número de redes y midiendo su energía, esta se distribuya de manera gaussiana en torno al cero.

Se crearon 10^7 redes y se midió su energía con el código que aparece debajo.

```
int main(){
    srand(time(NULL));
    FILE *f = fopen("InicializarRedTest.txt","w");
    long long int E, i = 0, nPasos = 10000000;
    for(i = 0; i<nPasos; i++){
        inicializarRed();
        E = energiaRed();
        fprintf(f,"%d\n",E);
        if(i % (nPasos / 100) == 0){
            printf("Progreso: %.1f%%\n",((double)i * 100) /nPasos);
        }
    }
    fclose(f);
    return 0;
}
```

A continuación se presenta la gráfica de frecuencias normalizada obtenida, 3.1, junto a su ajuste gaussiano, cuya bondad es evidente observando de la diferencia de órdenes entre las magnitudes y sus errores. El único parámetro con un error comparable es c, pero esto se explica entendiendo que su valor teórico debería ser exactamente 0, en este caso, un error del orden de 10^{-4} es razonablemente pequeño. Además notar que el valor de la desviación de la gaussiana es muy pequeño en comparación con el valor máximo de las energías, impulsando la efectividad del subalgoritmo.

$$g_{ajuste} = A \cdot e^{-\frac{(x-c)^2}{2 \cdot \sigma^2}}$$

A	$7{,}319364200 \cdot 10^{-3} \pm 3 \cdot 10^{-12}$
σ	$54{,}50821 \pm 2{,}4 \cdot 10^{-4}$
c	$-9{,}3 \cdot 10^{-4} \pm 2{,}4 \cdot 10^{-4}$

Tabla 1: Parámetros del ajuste gaussiano.

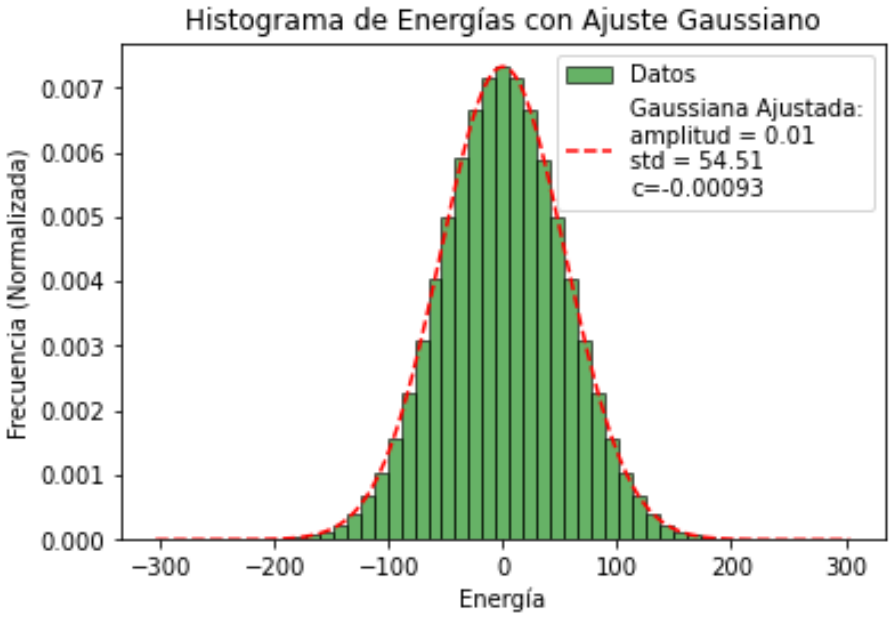


Figura 3.1: Histograma Energías

El siguiente test de verificación realizado se usó para comprobar el funcionamiento del algoritmo *TransfGauge*, que se muestra a continuación.

```
void TransfGauge(int pasos) {
int x,y,z;

for (int i = 0; i < pasos; i++)
    x = rand() % L;
    y = rand() % L;
    z = rand() % L;
    red[x][y][z][0]=-red[x][y][z][0];
    red[x][y][z][1]=-red[x][y][z][1];
    red[x][y][z][2]=-red[x][y][z][2];
    red[(x-1+L)%L][y][z][0]=-red[(x-1+L)%L][y][z][0];
    red[x][(y-1+L)%L][z][1]=-red[x][(y-1+L)%L][z][1];
    red[x][y][(z-1+L)%L][2]=-red[x][y][(z-1+L)%L][2];
}
```

Por definición, una transformación gauge se trata de una modificación de algún grado de libertad interno que no modifica ninguna propiedad física observable macroscópicamente. Entre las múltiples posibilidades, se decidió programar la transformación que, dado un punto de la red elegido aleatoriamente, invierte el signo de las seis aristas que lo rodean.

Es claro que esta modificación es Gauge, ya que no modifica la energía total del sistema. La energía total se calcula como la suma de las energías de cada plaqueta, la cual se calcula como el producto de las cuatro aristas que conforman cada plaqueta.

$$\sigma_p = \prod_{l \epsilon d} \sigma_l$$

Con el algoritmo TransfGauge, para cada una de las ocho plaquetas que tocan al punto escogido, se varían 2 de sus aristas.

$$\sigma'_p = \prod_{l \epsilon d} \sigma'_l = (-1)^2 \cdot \prod_{l \epsilon d} \sigma_l = (-1)^2 \cdot \sigma_p = \sigma_p$$

De ahí que el producto de sus aristas, que define su energía, no se vea alterado, y así que la energía total se mantenga constante.

Para comprobar que el código era adecuado se comprobó que la energía de la red no cambiaba al aplicar la transformación con TransfGauge, y efectivamente, con el código que aparece a continuación, se vio que la diferencia de energías era 0.

```
int main() {
    int i,j,k, E1,E2, pasos = 100000;
    srand(time(NULL));
    for (int cont=0; cont<10000;cont++){
            inicializarRed();
            E1=energiaRed();
            TransfGauge(1000);
            E2=energiaRed();
    printf("%d",E1-E2);}
    return 0;

}
```

4. Algoritmo de metrópolis y termalización

Para obtener la evolución de la red, simularemos su evolución mediante el algoritmo de Metrópolis. Este es un método de Monte Carlo que propone un cambio en el sistema (en este caso, en el espín de alguna de las aristas) que sucede con una cierta probabilidad en función de la temperatura y del cambio en la energía del sistema ΔE que esta alteración genera. Se implementará este algoritmo de la siguiente manera:

- Si $\Delta E \leq 0$, la probabilidad de aceptación del cambio es de 1.

- Si $\Delta E > 0$ la probabilidad de aceptación del cambio será de $e^{-\beta \Delta E}$ siendo $\beta = \frac{1}{k_B T}$ con k_B la constante de Boltzmann.

Evidentemente, cuanto mayor sea β (temperaturas bajas), más difícil será que se acepten cambios para cambios de energía positivos. Esto será algo que se deba ver para asegurar que el código funciona debidamente. Además, para reducir el costo computacional del programa, en vez de ejecutar *energiaRed* cada vez que se ejecuta el nuevo algoritmo de metrópolis, solo se calcularán las energías correspondientes a las plaquetas afectadas por el cambio, reduciendo así la cantidad necesaria de pasos significativamente.

Se programa el siguiente subalgoritmo:

```
void metropolis(int pasos, double beta) {
for (int i = 0; i < pasos; i++) {
    int x = rand() % L;
    int y = rand() % L;
    int z = rand() % L;
    int d = rand() % 3;

    int energia_inicial = 0;

    if(d==0){
        energia_inicial -= energiaPlaqueta(x, y, z, 0);
        energia_inicial -= energiaPlaqueta(x, y, z, 1);
        energia_inicial -= energiaPlaqueta(x, y, (z-1+L)%L, 0);
        energia_inicial -= energiaPlaqueta(x, (y-1+L)%L, z, 1);
    }
    else if(d==1){
        energia_inicial -= energiaPlaqueta(x, y, z, 1);
        energia_inicial -= energiaPlaqueta(x, y, z, 2);
        energia_inicial -= energiaPlaqueta((x-1+L)%L, y, z, 1);
        energia_inicial -= energiaPlaqueta(x, y, (z-1+L)%L, 2);
    }
    else if(d==2){
        energia_inicial -= energiaPlaqueta(x, y, z, 2);
        energia_inicial -= energiaPlaqueta(x, y, z, 0);
        energia_inicial -= energiaPlaqueta((x-1+L)%L, y, z, 0);
        energia_inicial -= energiaPlaqueta(x, (y-1+L)%L, z, 2);
    }

    red[x][y][z][d] *= -1;

    int energia_final = 0;
    if(d==0){
        energia_final -= energiaPlaqueta(x, y, z, 0);
        energia_final -= energiaPlaqueta(x, y, z, 1);
        energia_final -= energiaPlaqueta(x, y, (z-1+L)%L, 0);
        energia_final -= energiaPlaqueta(x, (y-1+L)%L, z, 1);
    }
    else if(d==1){
        energia_final -= energiaPlaqueta(x, y, z, 1);
        energia_final -= energiaPlaqueta(x, y, z, 2);
        energia_final -= energiaPlaqueta((x-1+L)%L, y, z, 1);
        energia_final -= energiaPlaqueta(x, y, (z-1+L)%L, 2);
    }
    else if(d==2){
        energia_final -= energiaPlaqueta(x, y, z, 2);
        energia_final -= energiaPlaqueta(x, y, z, 0);
        energia_final -= energiaPlaqueta((x-1+L)%L, y, z, 0);
        energia_final -= energiaPlaqueta(x, (y-1+L)%L, z, 2);
    }
```

```
        int deltaE = (energia_final - energia_inicial);
        //printf("Delta E = %d: ",deltaE);

        if (deltaE <= 0 || frand() < exp(-deltaE * beta)) {
            //printf("Cambio aceptado!\n");
        }
        else {
            red[x][y][z][d] *= -1;
            //printf("Cambio rechazado!\n");
        }
    }
}
```

La función *metropolis* genera 4 números aleatorios correspondientes a un punto de la red y una dirección en la que tomar la arista. Este entonces calcula la energía asociada a las aristas afectadas por el cambio, realiza el cambio en la arista (x, y, z, d), calcula el cambio de energía tras el cambio y el cambio de energía en general. Si el cambio se acepta, no se mantiene el cambio ya realizado y, en caso contrario, se devuelve a su valor original. El código pide un entero *pasos*, que indica cuántos cambios de la red debe proponer, y un número real *beta*, que simplemente es el valor de β utilizado para calcular las probabilidades.

Para la comprobación de este subalgoritmo, se generará la evolución de la energía para distintos betas en un número dado de pasos. Lo esperado será que, para temperaturas bajas (β pequeños), la caída de energía será más lenta que para temperaturas altas (β grande) donde se espera que prácticamente todos los cambios aceptados provoquen una disminución en al energía. Se utiliza entonces el siguiente código:

```
//Prueba metrópolis
int main() {
    srand(time(NULL));
    inicializarRed();
    //printf("Energía red: %d\n",energiaRed());
    int i = 0;
    double beta = 0.1;
    int nPasos = 10000;
    FILE *f = fopen("MetrópolisTest_beta0.1.txt","w");
    for (i = 0; i<nPasos; i++){
        fprintf(f,"%d\t %d\n", i, energiaRed());
        metropolis(1,beta);
        if(i % (nPasos / 10) == 0)
            printf("Progreso Termalizacion: %d%%\n", (1 * 100) / nPasos);
    }
    //printf("Energía final: %d\n",energiaRed());
    fclose(f);
}
```

Este código realiza la evolución de una red de distribución inicial aleatoria y escribe en un fichero el valor de la energía a cada paso de Metrópolis a temperatura fija *beta* que se da en la misma.

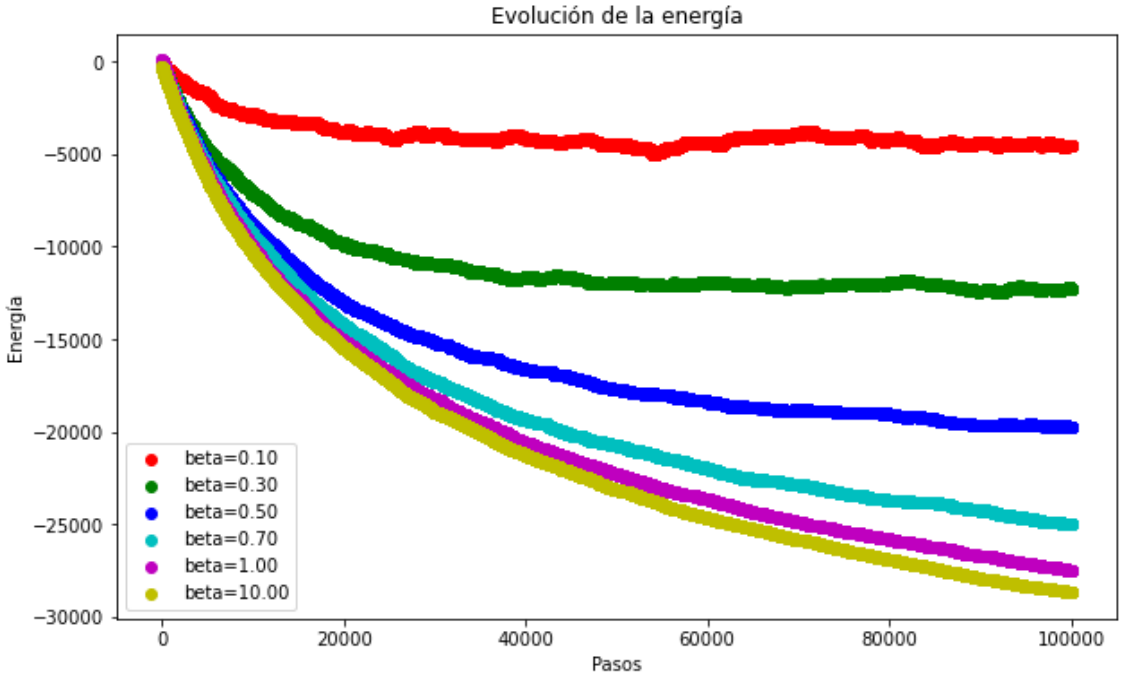


Figura 4.1: Evolución de la energía para distintos betas

La figura 4.1 muestra la evolución de la energía en redes aleatorias para distintos valores de β tras 10^5 pasos. Se ve claramente que, cuanto menor sea β, mayor es el valor al que tienden las evoluciones. De hecho, se puede ver claramente que las evoluciones de los 3 valores de β más grandes no alcanzan una situación de estabilidad, a pesar de que el decaimiento es mucho más rápido para las redes de mayor β, lo cual es de esperar ya que estos tenderán prácticamente a la energía mínima ($-3L^3$) para la cual se necesitan más pasos de los dados en la simulación para llegar.

El siguiente problema tiene que ver con esto: para las comprobaciones del cambio de fase de la red, se debe implementar una termalización de la red, es decir, se debe asegurar que los cambios de energía de la red van a ser prácticamente despreciables en el momento en el que se midan los valores de los espines. Esto implica realizar una cantidad suficiente de pasos de Montecarlo para que la red prácticamente no evolucione entre paso y paso.

Para obtener un criterio de termalización, se propone lo siguiente: inicialmente, se calculará la energía de una red aleatoria. Después, se realizará un número N de pasos de montecarlo y se calculará la energía de la red en este instante. Tras esto, se calculará la variación de la energía por paso:

$$\langle \Delta E \rangle = \frac{E_f - E_i}{N} \tag{2}$$

En el momento de la termalización, la energía se verá constante a un valor muy cercano al mínimo de energía salvo pequeñas fluctuaciones estadísticas, es decir, $\langle \Delta E \rangle \approx 0$. Por tanto, la simulación a realizar será una simulación de Montecarlo de un número muy grande de pasos divididos en intervalos de tamaño N para disintos valores de β para los que se calcula $\langle \Delta E \rangle$. La clave de esta simulación será ver en qué momento se puede afirmar que la variación es prácticamente nula para poder tomar este valor como el número de pasos de Montecarlo necesarios para asegurar que la

red ha termalizado. Como la simulación del siguiente apartado sólo se realizará para valores de β entre 0.7 y 0.8, solo se realizará la comprobación en torno a este rango de valores.

Se utiliza entonces el siguiente código:

```
//Test de termalización
int main(){
    srand(time(NULL));
    double beta = 0.68, beta_max=0.85;
    int nPasos = 1000000, i = 0, E1, E2;
    char filename[100];
    int intervalo = 5000, nMedidas = nPasos/intervalo;
    while (beta<=beta_max){
        inicializarRed();
        E1 = energiaRed();
        sprintf(filename,"MetropolisTest_beta%.2f.txt",beta);
        FILE *f = fopen(filename,"w");
        printf("Fichero beta = %lf\n",beta);
        for(i = 0; i<nMedidas; i++){
            metropolis(intervalo,beta);
            E2 = energiaRed();
            fprintf(f,"%d \t%d\n",intervalo*i,E2-E1);
            E1 = E2;
            if(i % (nMedidas / 10) == 0)
                printf("Progreso Termalizacion:
                    %d%%\n", (i * 100) / nMedidas);
        }
        beta = beta+0.02;
    }
}
```

Este algoritmo recrea lo descrito anteriormente fijando $N = 5000$ (llamado *intervalo* en el código, el número total de iteraciones (llamado $nPasos$) en 10^6 (200 separaciones en total) y para β entre 0.68 y 0.84 en saltos de 0.2. Notar que este calcula la variación de energía en bruto pero esto se arregla dividiendo todos los valores entre N en el código de Python para la representación gráfica:

La figra 4.2 muestra el espectro completo de la simulación. Evidentemente, esta es muy difícil de comprender al haber una gran cantidad de datos acumulados en muy poco espacio pero da una buena idea de lo que sucede. Se ve que, en todas las simulaciones, la tendencia a 0 y de carácter inverso al que se ve en la figura 4.1 como era de esperar y además todas las tendencias son muy similares sin importar el valor de β. Esto es debido a que el rango de valores utilizado es muy reducido, luego es de esperar que estos efectos no afecten demasiado a la velocidad de evolución del código.

La figura 4.3 muestra el espectro reducidos a pasos entre $5{,}5 \cdot 10^5$ y $6{,}5 \cdot 10^5$. En esta, se ve que, salvo datos anómalos, $\langle \Delta E \rangle$ está comprendida entre -0.03 y 0.02, es decir, para que la energía varíe en una cantidad de como mucho 3 unidades se necesitan por lo menos 100 pasos. Por la geometría de la red, un cambio en una arista muestra un cambio de energías de 0, 4 u 8, luego se necesitan cientos de pasos para

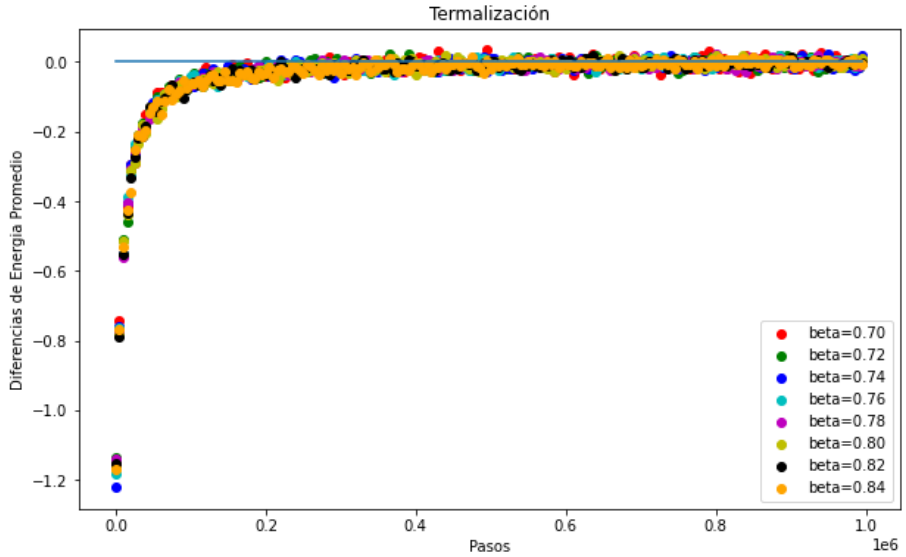


Figura 4.2: Espectro completo de diferencia promedio de energías.

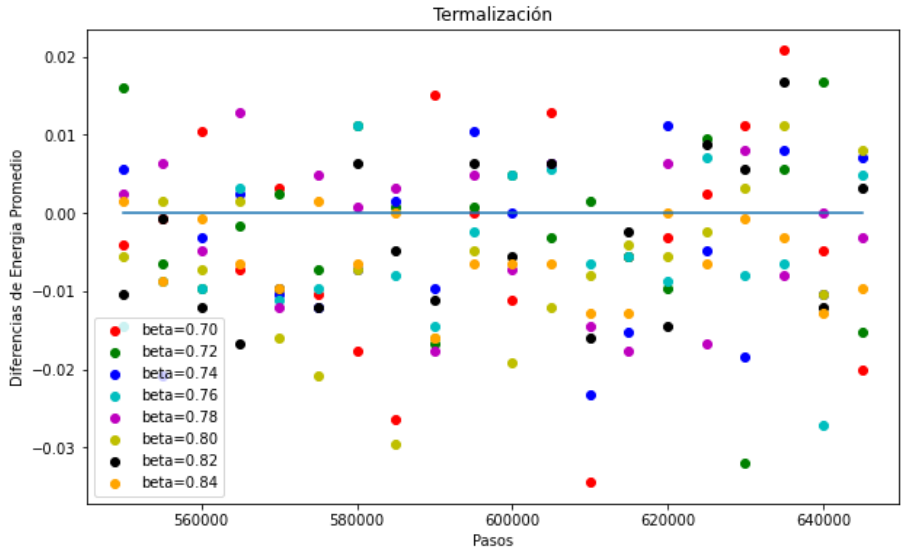


Figura 4.3: Espectro reducido de diferencia promedio de energías.

que se vea algún tipo de diferencia que, seguramente, acabe siendo insignificante. Además, hay una buena relación entre cambios de energía positivos y negativos por lo que, llegados a esta cantidad de pasos, es muy seguro que toda variación en la energía sea debido a fluctuaciones estadísticas. Por tanto, consideraremos que la red ha termalizado una vez se hayan recorrido $6 \cdot 10^5$ pasos.

5. Comprobación de las leyes del área y el perímetro

Por último queremos observar el comportamiento de nuestro sistema estudiando el valor esperado de los loops de Wilson, $\langle W_{nn} \rangle$ (productos de espines en curvas cerradas de tamaño $n \times n$), para diferentes valores de β.

Sabemos por teoría que el modelo tiene una transición de fase para $\beta_c \approx 0,7613$. Esto es, un cambio en el comportamiento macroscópico del sistema al superar un

valor crítico de un parámetro, en este caso la temperatura. De este modo, cuando $\beta < \beta_c$, esperamos observar una tendencia exponencial con el área:

$$\langle W_{nn} \rangle \propto \exp(-\sigma nn)$$

Mientras que cuando $\beta > \beta_c$, la tendencia esperada va con el perímetro $(4n)$.

Para implementar el código creamos los siguientes subalgoritmos:

```
    int Loops(int n, int x, int y, int z, int d){
    if(n >= L){
        printf("Demasiado grande");
        return 0;
    }
    else{
        int i, energia = 1;
        if(d==0){
            for(i = 0; i<n; i++){
                energia *= red[x][y][(z+i)%L][2]*
                red[(x+i)%L][y][z][0]*
                red[(x+n)%L][y][(z+i)%L][2]*
                red[(x+i)%L][y][(z+n)%L][0];
            }
        }
        else if(d==1){
            for(i = 0; i<n; i++){
                energia *= red[(x+i)%L][y][z][0]*
                red[(x+n)%L][(y+i)%L][z][1]*
                red[x][(y+i)%L][z][1]*
                red[(x+i)%L][(y+n)%L][z][0];
            }
        }
        else if (d==2){
            for(i = 0; i<n; i++){
                energia *= red[x][y][(z+i)%L][2]*
                red[x][(y+i)%L][z][1]*
                red[x][(y+n)%L][(z+i)%L][2]*
                red[x][(y+i)%L][(z+n)%L][1];
            }
        }
    return energia;
    }
}

void LoopsHastaNmax(int nMin, int nMax, int *ELoops,
        int x, int y, int z, int d){
    int i;
    for(i=0; i<=nMax-nMin; i++){
        ELoops[i] = Loops(nMin+i,x,y,z,d);
    }
}
```

Loops calcula la energía del loop cuadrado de tamaño $n \times n$ partiendo del punto (x, y, z) en el plano marcado por la dirección d como el producto de las aristas que componen ese loop.

LoopsHastaNMax se sirve del subalgoritmo anterior para, partiendo del mismo punto (x, y, z) y en el mismo plano, asignado por la dirección d, llenar un array con las energías de los loops de tamaño $n \times n$ con n desde n_{mn} hasta n_{mx}.

El código implementado se compone de un bucle de tamaño el número de configuraciones a medir, que para tener estadística, tomamos el valor $nRedes = 100{,}000$ donde en cada iteración realizamos lo que explicamos a continuación. En primer lugar usamos inicializarRed y termalizamos usando el valor que se ha argumentado en la sección anterior. Ya termalizado, recorremos todos los puntos de la red, y calculamos con LoopsHastaNMax la energía de los loops con tamaños desde $n_{mn} = 1$ hasta $n_{mx} = 10$ en las tres direcciones del espacio. Tras haber procesado todas las aristas, calculamos el promedio de los valores de energía de cada tamaño de loop y su desviación estándar.

En el anexo B adjuntamos el código, que ejecutamos para distintos valores de β y con los cuales realizamos el estudio de las leyes del área y el perímetro, cuyos resultados se presentan a continuación.

El análisis se realizará para los valores de $\beta = 0{,}72, 0{,}74, 0{,}80$, obteniéndose la media y la varianza de las energías de los loops de tamaños entre 1 y 10 para un total de 100000 redes con $L = 24$. No se estudian loops más grandes ya que, a partir de cierto tamaño, se empieza a perder la cobertura de las energías de los loops. Después, se utilizará un código en Python para la obtención de las gráficas y los ajustes funcionales. Los detalles sobre el tratamiento tratamiento estadístico del error aparecen en el anexo A mientras y el código de Python realizado para los ajustes aparecerá en el anexo C.

β	**Factor de proporcionalidad** (a)	**Parámetro de red** (σ)
0.72	0.9680 ± 0.0025	-0.2289 ± 0.0014
0.74	0.9710 ± 0.0020	-0.2073 ± 0.0010
0.80	1.61 ± 0.10	-0.162 ± 0.012

Tabla 2: Parámetros de ajuste

La tabla 2 muestra los valores de los parámetros de ajuste junto con su error mientras que las figuras 5.1, 5.2 y 5.3 lo representan gráficamente. En el anexo D, aparece una tabla con los valores exactos de cada uno de los puntos junto con su error.

Como se observa, en los casos de $\beta = 0,72$ y $\beta = 0,74$ la ley del área se ajusta considerablemente bien. Sin embargo, para el caso de $\beta = 0,80$, donde se esperaba que se cumpliera la ley del perímetro, el ajuste no llega a ser del todo bueno. Una de las razones más obvias es que las dimensiones de la red sean finitas, de hecho son bastante pequeñas (24x24x24). Ambas leyes son asíntoticas, es decir, se deberían cumplir para una red infinita. No hubiera sido de extrñar que la ley del área tampoco se hubiese ajustado bien. Puede ser interesante analizar redes más grandes y en mayor número si se realiza una repetición de este experimento.

Por otro lado, en los errores se tiene que los loops de mayor dimensión poseén un mayor error, ya que hay poca cobertura para estos últimos. En cuanto a los valores

negativos, comentar que estos carecen de sentido según las leyes que se han obtenido y se deben al error que poseén los valores obtenidos.

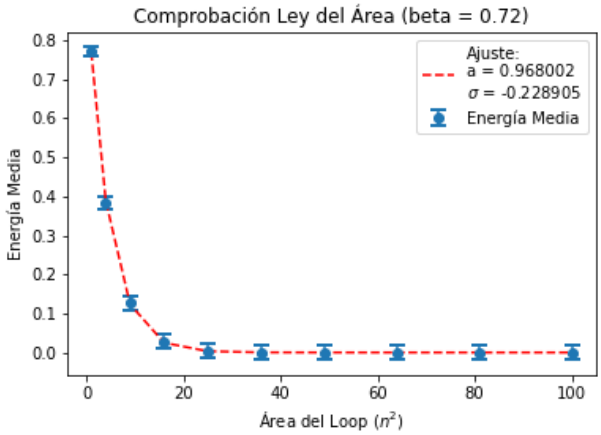


Figura 5.1: Comprobación de la ley del área ($\beta = 0{,}72$)

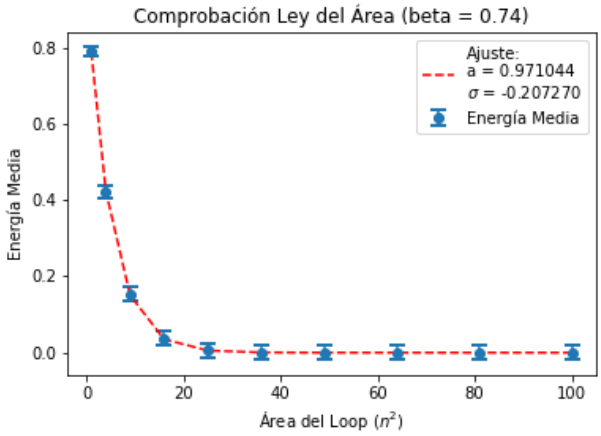


Figura 5.2: Comprobación de la ley del área ($\beta = 0{,}74$)

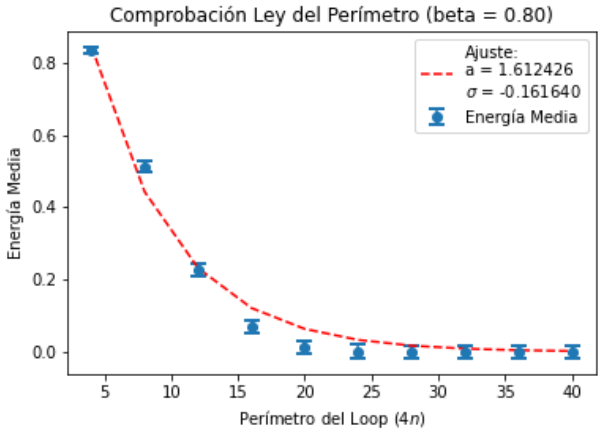


Figura 5.3: Comprobación de la ley del perímetro ($\beta = 0{,}80$)

6. Conclusiones

A lo largo de la práctica se ha estudiado el modelo de Ising Gauge tridimensional usando simulaciones de Monte Carlo con el objetivo de comprobar las leyes del área y del perímetro para loops de Wilson bajo diferentes condiciones. Para ello se han creado funciones e ingeniado métodos para comprobarlas que han permitido que se verificasen con éxito a lo largo de la práctica.

Los resultados obtenidos muestran un buen ajuste para valores de β inferiores al punto crítico, cuya tendencia va con el área, y un ajuste razonable para el valor superior, con la ley del perímetro. Las razones aportadas para esto, relacionadas con la falta de estadística por las dimensiones de la red, son coherentes y podrían ser verificadas con una mayor capacidad de computación. Para el ajuste se ha implementado un método para la estimación de errores correlacionados que ofrece un balance entre precisión y eficiencia computacional.

Con esto, se puede concluir que se han cumplido los objetivos, obteniendo resultados que permiten validar el modelo.

A. Tratamiento del error:

En un caso general de medidas sin correlación, el error en la medida se puede calcular como:

$$\sigma^2 = \frac{1}{\left(\sum_{i=1}^{N} \frac{1}{\sigma_i^2}\right)} \tag{3}$$

siendo N el número total de medidas realizadas y σ_i el error en la medida de cada red. El valor σ_i podemos calcularlo usando que:

$$\sigma_i^2 = \langle E^2 \rangle - \langle E \rangle^2 \tag{4}$$

donde en el que caso particular de esta simulación $E^2 = 1$ siempre ya que esta medida solo puede tomar los valores 1 y -1.

Sin embargo, al depender muchos loops de las mismas aristas y, por tanto, de las mismas medidas, las medidas de las energías de todos los loops de una misma red están correlacionadas, por lo que este tratamiento del error podría subestimar el error en la medida.

Formalmente, este cálculo se debería de realizar utilizando matrices de covariancia. Sin embargo, al estar trabajando con redes de tamaño 24 y un total de cien mil redes por medición, es excesivamente costoso realizar dicho cálculo ya que, sin tener esto en cuenta, la ejecución de la simulación lleva un mínimo de 3 horas en los ordenadores empleados. Si se debe realizar aproximadamente cien mil medidas más para esto, el factor de tiempo se vuelve inmanejable. Por tanto, se propone la siguiente aproximación:

Supondremos que los efectos de la correlación son aproximadamente iguales en todas las medidas de los loops de un mismo tamaño. Esto es lógico ya que los loops del mismo tamaño dependen del mismo número de aristas. De esta forma, podremos escribir el error como:

$$\sigma_{eff} = f \cdot \sigma \tag{5}$$

siendo σ el error estimado en ausencia de correlación y f un factor de ajuste. De esta manera, se obtiene un cálculo de error sencillo y computacionalmente eficiente ya que sólo depende de la estimación de un único factor.

Sólo quedaría la pregunta de qué factor es correcto. Para ello, se realizará el análisis formal de covarianzas para una única red de cada valor de β estudiado y se comparará el cociente del error teniendo en cuenta la covarianza y no teniéndola. Para estos cálculos, se asumirá correlaciones máximas para tener el máximo error posible ($\rho = 1$).

La metodología del código es la siguiente: se realiza una medida de las medias de energías junto con su desviación para cada tamaño de los loops pero dividido en pequeños grupos (en este caso, de $24 \cdot 24$ medidas, es decir, cada plano del eje x de la red). La implementación es esencialmente la misma que la del apartado anterior añadiendo una suma parcial que calcule después la energía media y el error en la medida de cada plano. Con el parámetro de correlación, se calcula la matriz de covarianza entre las medidas y, de esta, se estima el error utilizando:

$$\sigma_{eff,i}^2 = \frac{1}{\left(\sum_{j=1}^{N} \frac{1}{C_{ij}}\right)} \tag{6}$$

Finalmente, se calcula el cociente entre estos con los obtenidos para toda la red y se aplica el máximo de estos como factor de cobertura para toda la práctica.

Tras realizar el análisis para 3 redes con los valores de beta del apartado 5, todos coinciden en que su máximo está en torno a 5.7. Es por eso que se tomará durante todo este apartado:

$$f = 5{,}7$$

por lo que todos los errores se verán aumentados como corrección de correlación de los parámetros.

Se adjunta el código de Python implementado para tomar esta decisión:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Leer los datos del fichero
filename = 'C:/Users/ricar/Desktop/Pruebas VSC/Prubea/P7/Error_beta0.80.txt'
data = np.loadtxt(filename)

# Extraer columnas
red_number = data[:, 0]
loop_size = data[:, 1]
mean_energy = data[:, 2]
std_energy = data[:, 3]

# Calcular el valor medio y el error de las energías
unique_n = np.unique(loop_size)
avg_energy = []
error_energy = []

for n in (unique_n):
    indices = np.where(loop_size == n)
    avg_energy.append(np.mean(mean_energy[indices]))

def covariance_matrix(std_devs, rho):
    """Crea una matriz de covarianza basada en una correlación común rho."""
    n = len(std_devs)
    cov_matrix = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            if i == j:
                cov_matrix[i, j] = std_devs[i]**2
            else:
                cov_matrix[i, j] = rho * std_devs[i] * std_devs[j]
    return cov_matrix

# Cálculo de f:
indices = np.where(red_number != 25)
rho = 1  # Correlación común
cov_matrix = covariance_matrix(std_energy[indices], rho)
```

```
jndices = np.where(red_number == 25)
aux=[]
sigma_0 = std_energy[jndices]
sigma_eff = []

for i in range (len(cov_matrix[:][0])):
    aux.append(np.sqrt(np.sum(cov_matrix[i][indices])))

aux = np.array(aux)

for n in (unique_n):
    indices = np.where(np.logical_and(loop_size == n, red_number != 25))
    sigma_eff.append(np.sum(np.sqrt(1/(sum(1/(aux[indices]**2))))))

f = sigma_eff/sigma_0

print(f)
```

B. Código Comprobación Ley del Área y del Perímetro

```
int main(){
    srand(time(NULL));
    int nMin = 1, nMax = 10;
    int control = nMax-nMin+1;
    int ELoops[control], i, j, x, y, z, d;
    int nRedes = 100000;
    int N = L*L*L*3;
    int tot = N*control;
    double avg[control], std[control];
    int suma[control];
    double beta = 0.8;
    char filename[100];
    FILE *f;
    sprintf(filename,"LoopsRed%d_beta%.2f.txt",L,beta);
    f = fopen(filename,"w");

    for(i=0; i<nRedes; i++){

        for(j=0; j<control; j++){
            suma[j] = 0;
        }

        inicializarRed();
        metropolis(600000,beta);

        for(x=0; x<L; x++){
            for(y=0; y<L; y++){
                for(z=0; z<L; z++){
                    for(d=0; d<3; d++){
                        LoopsHastaNmax(nMin,nMax,ELoops,x,y,z,d);

                        for(j=0; j<control; j++){
                            suma[j] += ELoops[j];
                        }
                    }
                }
            }
        }
        for(j=0; j<control; j++){
            avg[j] = ((double) suma[j])/((double) N);
            std[j] = sqrt((1.0 - avg[j]*avg[j]))/sqrt(((double) tot/(double) N));
        }
        if(i % (nRedes / 1000) == 0)
            printf("Progreso: %d%%\n", (i * 100) / nRedes);
    }
    fclose(f);}
```

C. Código de Python utilizado para el ajuste funcional:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Definir el rango de valores de beta
betas = [0.72, 0.74, 0.80]  # Lista de valores de beta

# Factor de escala
f = 5.7

# Función de ajuste
def exponencial(x, a, b):
    return a * np.exp(b * x)

# Iterar sobre los valores de beta
for beta in betas:
    # Leer los datos correspondientes a cada beta
    filename = f'C:/Users/ricar/Desktop/Pruebas VSC/
        Prubea/P7/LoopsRed24_beta{beta:.2f}.txt'
    data = np.loadtxt(filename)

    # Extraer columnas
    red_number = data[:, 0]
    loop_size = data[:, 1]
    mean_energy = data[:, 2]
    std_energy = data[:, 3]*f

    # Calcular el valor medio y el error de las energías
    unique_n = np.unique(loop_size)
    avg_energy = []
    error_energy = []

    for n in unique_n:
        indices = np.where(loop_size == n)
        avg_energy.append(np.mean(mean_energy[indices]))
        error_energy.append(np.sqrt(1 / (np.sum(1 / std_energy[indices]**2))))

    # Convertir listas a arrays
    avg_energy = np.array(avg_energy)
    error_energy = np.array(error_energy)

    # Ajuste de datos según beta
    if beta < 0.76:
        ua = unique_n**2
        xlabel = 'Área del Loop ($n^2$)'
        title = 'Comprobación Ley del Área'
```

```
else:
    ua = unique_n * 4
    xlabel = 'Perímetro del Loop ($4n$)'
    title = 'Comprobación Ley del Perímetro'

# Ajuste con la función exponencial
ppot, pcov = curve_fit(exponencial, ua, avg_energy, p0=(1, -1),
        sigma=error_energy)

# Crear gráfico
plt.errorbar(ua, avg_energy, yerr=error_energy, fmt='o', capsize=5,
    capthick=2, label='Energía Media')
plt.plot(ua, exponencial(ua, *ppot), 'r--',
         label=f'Ajuste:\na = {ppot[0]:.6f}\n$\sigma$ = {ppot[1]:.6f}')
plt.xlabel(xlabel)
plt.ylabel('Energía Media')
plt.title(f'{title} (beta = {beta:.2f})')
plt.legend()
plt.savefig(f'C:/Users/ricar/Desktop/Pruebas VSC/Prubea/P7/
    Plot_beta{beta:.2f}.png')
plt.close()

# Guardar resultados en archivo
output_filename = f'C:/Users/ricar/Desktop/Pruebas VSC/Prubea/P7/
    Resultados_beta{beta:.2f}.txt'
with open(output_filename, 'w') as file:
    file.write("n\tE\n")
    for i in range(len(unique_n)):
        file.write(f"{unique_n[i]}\t{avg_energy[i]:.6f} ±
            {error_energy[i]:.6f}\n")
    file.write(f"\nParámetros del ajuste:\n")
    file.write(f"a = {ppot[0]:.6f} ± {np.sqrt(pcov[0][0]):.6f}\n")
    file.write(f"b = {ppot[1]:.6f} ± {np.sqrt(pcov[1][1]):.6f}\n")

print(f"Resultados para beta = {beta:.2f} guardados en {output_filename}")
```

D. Datos exactos de las figuras:

	E		
n	$\beta = 0{,}72$	$\beta = 0{,}74$	$\beta = 0{,}80$
1.0	0.770516 ± 0.011486	0.789806 ± 0.011052	0.834159 ± 0.009937
2.0	0.383893 ± 0.016641	0.420473 ± 0.016351	0.510811 ± 0.015492
3.0	0.127276 ± 0.017877	0.153692 ± 0.017809	0.227940 ± 0.017548
4.0	0.027259 ± 0.018018	0.037149 ± 0.018012	0.070526 ± 0.017979
5.0	0.003622 ± 0.018025	0.005648 ± 0.018024	0.014349 ± 0.018022
6.0	0.000301 ± 0.018025	0.000492 ± 0.018025	0.001846 ± 0.018025
7.0	0.000006 ± 0.018025	0.000032 ± 0.018025	0.000110 ± 0.018025
8.0	-0.000010 ± 0.018025	0.000007 ± 0.018025	-0.000041 ± 0.018025
9.0	0.000015 ± 0.018025	-0.000013 ± 0.018025	0.000033 ± 0.018025
10.0	-0.000025 ± 0.018025	0.000023 ± 0.018025	0.000018 ± 0.018025

Tabla 3: Datos energéticos obtenidos